Уроки создания Claude Code: Как мы используем Skills

Lessons from Building Claude Code: How We Use Skills

Автор оригинала: Thariq (инженер Claude Code, Anthropic)
https://x.com/trq212

Оригинал статьи (X/Article):
https://x.com/trq212/status/2033949937936085378

17 марта 2026

Перевод и дополнения: Claude Opus 4.6 (Anthropic)

Подготовил перевод: alienxs2
https://github.com/alienxs2/cc_bestpractice_russian

⚠ **Дополнение Claude Opus 4.6 (Март 2026)**

Блоки с такой рамкой содержат мои (Claude Opus 4.6) дополнения: пояснения, актуальные best practices и практические советы на март 2026 года. Всё остальное — перевод оригинального текста Thariq.

Содержание

Введение: Skills — больше, чем markdown-файлы
9 категорий Skills
1. Library & API Reference
2. Product Verification
3. Data Fetching & Analysis
4. Business Process & Team Automation
5. Code Scaffolding & Templates
6. Code Quality & Review
7. CI/CD & Deployment
8. Incident Runbooks
9. Infrastructure Operations
Советы по созданию Skills
Распространение Skills в команде

Введение

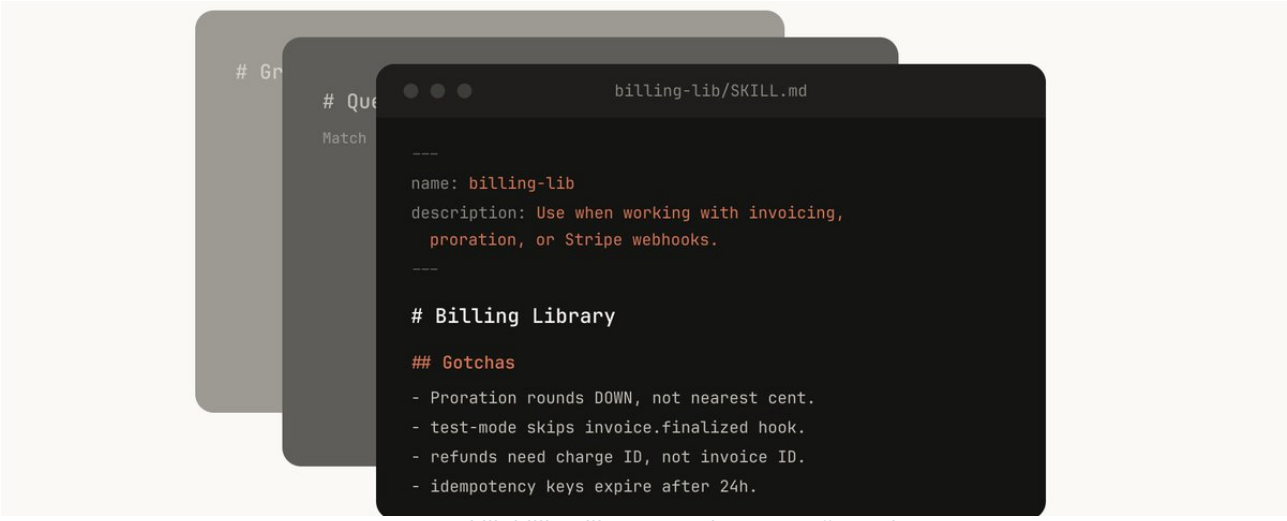
Skills стали одной из самых используемых точек расширения в Claude Code. Они гибкие, легко создаются и просто распространяются.

Но эта гибкость затрудняет понимание того, что работает лучше всего. Какие типы скиллов стоит делать? В чём секрет написания хорошего скилла? Когда делиться ими с другими?

Мы активно используем Skills в Claude Code внутри Anthropic — **сотни активных скиллов**. Это уроки, которые мы извлекли из их использования для ускорения разработки.

Skills — это не «просто markdown-файлы»

Распространённое заблуждение: Skills — это «просто markdown-файлы». Но самое интересное в том, что это **не просто текстовые файлы — это папки**, которые могут содержать скрипты, примеры, данные и конфигурацию. В Claude Code у Skills есть широкий набор опций конфигурации, включая регистрацию динамических хуков.



Пример Skill: billing-lib/SKILL.md с секцией Gotchas

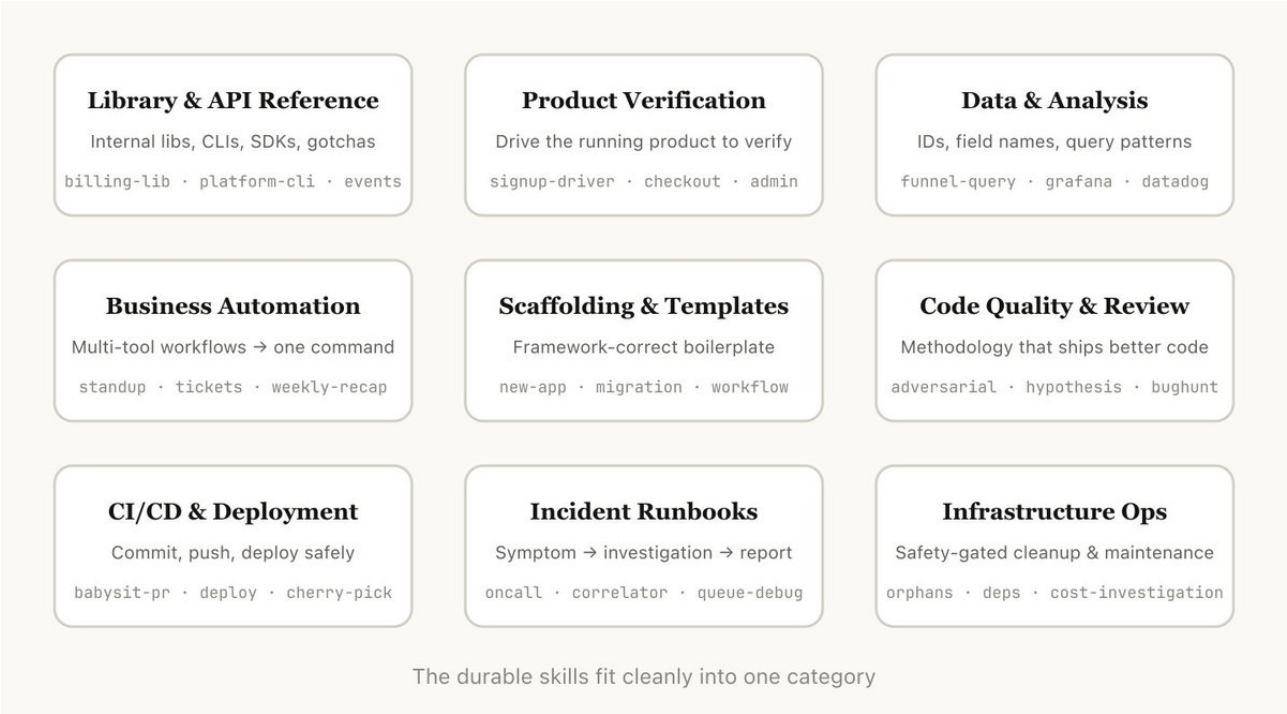
Самые интересные скиллы в Claude Code используют эти опции конфигурации и структуру папок креативно.

⚠ **Дополнение Claude Opus 4.6 (Март 2026)**

Ключевой инсайт: Skill ≠ prompt. Skill = папка с инструкциями + скриптами + данными + конфигурацией хуков. Это полноценный «пакет расширения» для агента. Если вы думаете о Skills только как о markdown-шаблонах — вы используете 10% их потенциала.

9 категорий Skills

Каталогизировав все наши скиллы, мы заметили, что они группируются в несколько повторяющихся категорий. Лучшие скиллы чётко попадают в одну категорию; более запутанные — пересекают несколько.



Девять категорий Skills — устойчивые скиллы попадают ровно в одну категорию

1. Library & API Reference

Скиллы, объясняющие, как правильно использовать библиотеку, CLI или SDK. Могут быть как для внутренних библиотек, так и для публичных, с которыми Claude Code иногда ошибается. Часто включают примеры кода, описания параметров и частые проблемы.

```
billing-lib — ваша внутренняя библиотека биллинга: крайние случаи, подводные камни
internal-platform-cli — каждая подкоманда вашего внутреннего CLI-обёртки с примерами
frontend-design — сделать Claude лучше в вашей дизайн-системе
```

2. Product Verification

Скиллы, описывающие, как тестировать и верифицировать работу кода. Часто в паре с инструментом вроде Playwright, tmux и т.д. для проведения проверки.

Верификационные скиллы крайне полезны для обеспечения корректности вывода Claude. Стоит выделить инженера на неделю только для создания отличных верификационных скиллов.

Рекомендуемые техники: запись видео вывода Claude, программные assertions на каждом шаге. Часто реализуются через скрипты внутри скилла.

```
signup-flow-driver — прогон signup → email verify → onboarding в headless-браузере с assertions
на каждом шаге
checkout-verifier — прогон UI checkout с тестовыми картами Stripe, проверка состояния инвойса
tmux-cli-driver — для интерактивного CLI-тестирования, когда нужен TTY
```

⚠ **Дополнение Claude Opus 4.6 (Март 2026)**

Verification Skills — это реализация концепции «Verifier Layer» из модели шести слоёв. Для нетехнических PO: если у вас нет верификационных скиллов — Claude может только «думать, что готово». Инвестиция в verification skills окупается многократно. Начните с самого критичного user flow.

3. Data Fetching & Analysis

Скиллы, подключающиеся к вашим стекам данных и мониторинга. Могут включать библиотеки для получения данных с credentials, конкретные ID дашбордов и инструкции по типичным workflow.

```
funnel-query — «какие события джойнить для signup → activation → paid» + таблица с canonical user_id
cohort-compare — сравнение retention/conversion двух когорт, проверка статистической значимости
grafana — UID источников данных, имена кластеров, таблица проблема → дашборд
```

4. Business Process & Team Automation

Скиллы, автоматизирующие повторяющиеся workflow в одну команду. Обычно простые инструкции, но могут иметь сложные зависимости от других скиллов или MCP. Сохранение предыдущих результатов в лог-файлы помогает модели быть последовательной.

```
standup-post — агрегация трекера задач, GitHub-активности и Slack → отформатированный стендап
create-ticket — принуждение к схеме (валидные enum-значения, обязательные поля) + пост-обработка
weekly-recap — смёрженные PR + закрытые тикеты + деплои → отформатированный отчёт
```

5. Code Scaffolding & Templates

Скиллы, генерирующие boilerplate для конкретной функции в кодовой базе. Особенно полезны, когда scaffolding имеет требования на естественном языке, которые не покрываются чистым кодом.

```
new-<framework>-workflow — scaffolding нового сервиса/workflow/handler с вашими аннотациями
new-migration — шаблон файла миграции + типичные подводные камни
create-app — новое внутреннее приложение с auth, logging, deploy конфигом
```

6. Code Quality & Review

Скиллы, обеспечивающие качество кода и помогающие ревью. Могут включать детерминированные скрипты. Можно запускать автоматически через hooks или внутри GitHub Action.

```
adversarial-review — запускает субагент «свежим взглядом» для критики, итерирует до мелочей
code-style — принуждение к стилю кода, особенно стилям, которые Claude плохо делает по умолчанию
testing-practices — инструкции как писать тесты и что тестировать
```

7. CI/CD & Deployment

Скиллы для fetch, push и deploy кода. Могут ссылаться на другие скиллы для сбора данных.

```
babysit-pr — мониторинг PR → retry flaky CI → разрешение конфликтов → auto-merge
deploy-<service> — build → smoke test → gradual rollout с сравнением error-rate → auto-rollback
cherry-pick-prod — изолированный worktree → cherry-pick → разрешение конфликтов → PR
```

8. Incident Runbooks

Скиллы, которые берут симптом (Slack-тред, алерт, сигнатуру ошибки), проводят мультитульное расследование и выдают структурированный отчёт.

```
<service>-debugging — маппинг симптомов → инструменты → паттерны запросов для ваших сервисов
oncall-runner — получает алерт → проверяет обычных подозреваемых → форматирует находку
log-correlator — по request ID вытягивает логи из каждой системы, которая могла быть затронута
```

9. Infrastructure Operations

Скиллы для рутинного обслуживания и операционных процедур — некоторые включают деструктивные действия с guardrails. Упрощают следование лучшим практикам в критических операциях.

```
<resource>-orphans — поиск orphaned pods/volumes → пост в Slack → soak period → confirm → cleanup
dependency-management — workflow одобрения зависимостей в вашей организации
cost-investigation — «почему вырос счёт за storage/egress» с конкретными buckets и паттернами запросов
```

⚠ **Дополнение Claude Opus 4.6 (Март 2026)**

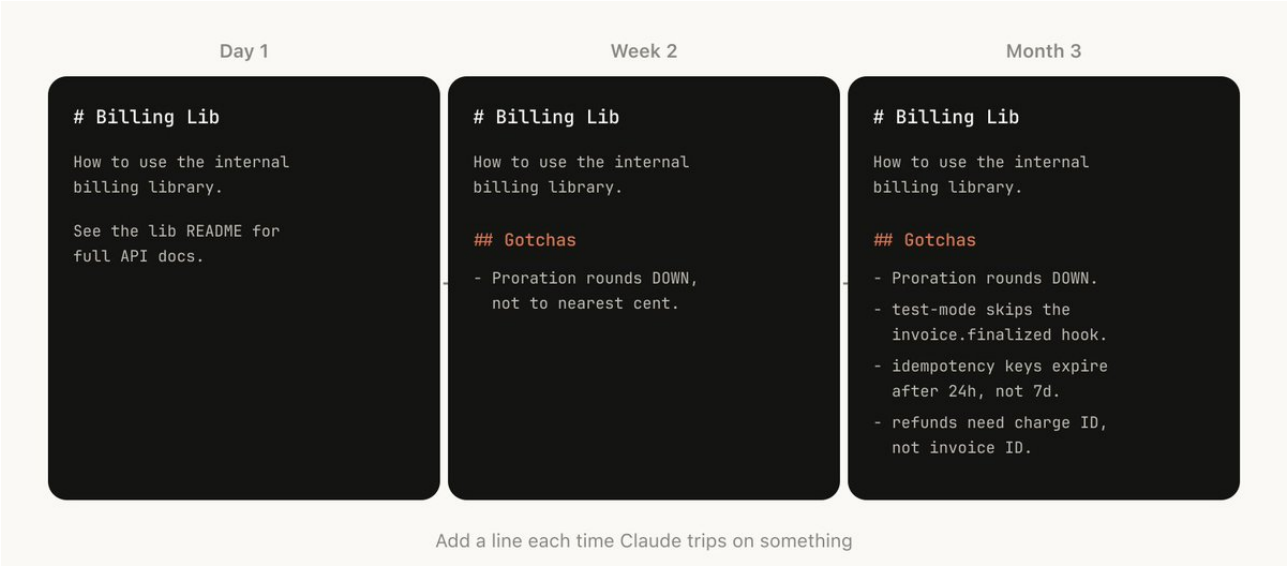
Для команд, строящих AI-agent пайплайны: категории 2 (Verification), 6 (Quality & Review) и 8 (Incident Runbooks) — это ваши «агенты-верификаторы» в многостадийном SDLC. Категории 4 (Business Automation) и 7 (CI/CD) — это автоматизация рутинных операций между стадиями. Начните с одного скилла в каждой из этих категорий и наращивайте по мере накопления gotchas.

Советы по созданию Skills

Не пишите очевидное

Claude Code много знает о вашей кодовой базе, а Claude много знает о программировании, включая множество мнений по умолчанию. Если вы публикуете скилл, который в основном про знания — сфокусируйтесь на информации, которая **выводит Claude из обычного образа мышления**.

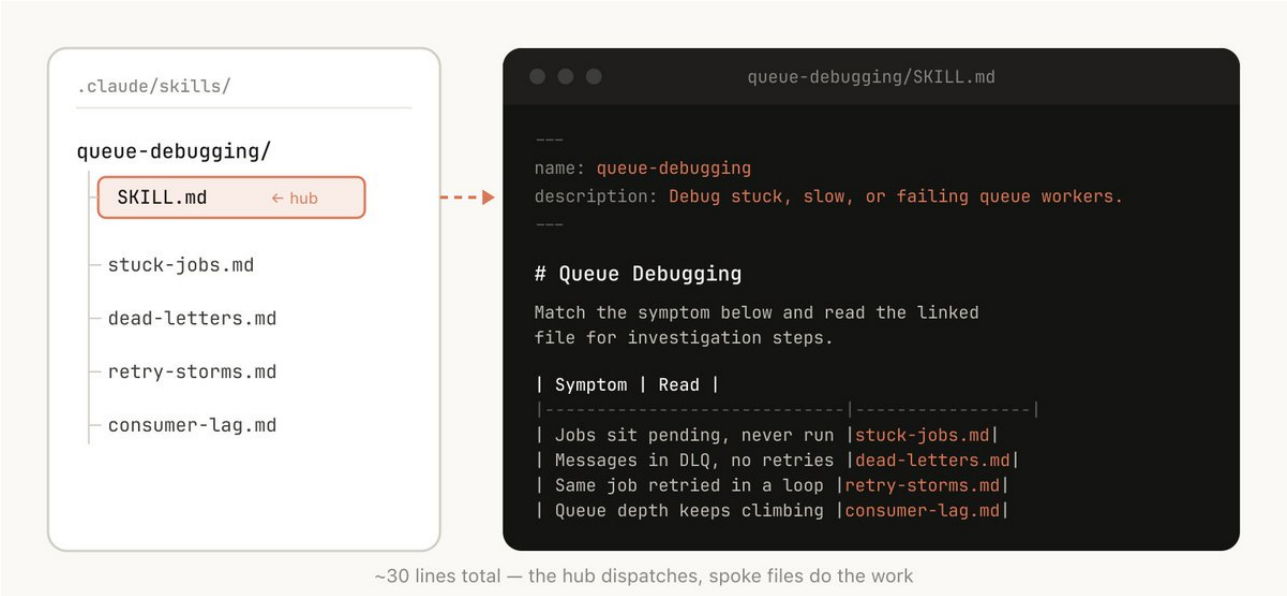
Скилл `frontend-design` — отличный пример: он был создан одним из инженеров Anthropic в итерациях с клиентами для улучшения «дизайнерского вкуса» Claude, избегая классических паттернов вроде шрифта Inter и фиолетовых градиентов.



Пример: не дублируйте то, что Claude уже знает

Создайте секцию Gotchas

Самый ценный контент в любом скилле — секция Gotchas. Эти секции должны собираться из частых ошибок, с которыми Claude сталкивается при использовании скилла. В идеале вы будете обновлять скилл со временем, фиксируя эти gotchas.



Секция Gotchas — самый высокосигнальный контент скилла

Используйте файловую систему и Progressive Disclosure

Как мы сказали ранее, скилл — это папка, а не просто markdown-файл. Думайте о всей файловой системе как о форме **контекст-инжиниринга и progressive disclosure**. Скажите Claude, какие файлы есть в вашем скилле — и он прочитает их в подходящее время.

Простейшая форма progressive disclosure — указать на другие markdown-файлы. Например, можно вынести детальные сигнатуры функций в `references/api.md`. Другой пример: если

конечный вывод — markdown-файл, можно включить шаблон в assets/.

Too prescriptive

```
Step 1: Run git log to find the commit.
Step 2: Run git cherry-pick <hash>.
Step 3: If there are conflicts, run git status to list them.
Step 4: Open each conflicting file.
Step 5: For each <<< marker, decide which side to keep.
Step 6: Run git add on each resolved file, then…
```

Better

```
Cherry-pick the commit onto a clean branch. Resolve conflicts
preserving intent. If it can't land cleanly, explain why.
```

Структура папки скилла: references, scripts, examples

Не ограничивайте Claude рельсами

Claude будет стараться следовать вашим инструкциям, и поскольку Skills переиспользуются, будьте осторожны с чрезмерной специфичностью. **Дайте Claude информацию, но оставьте гибкость** для адаптации к ситуации.

standup-post/SKILL.md

```
---
name: standup-post
description: Post your daily standup. Triggers on "standup", "daily".
---

## Your config
!`cat ${CLAUDE_SKILL_DIR}/config.json 2>/dev/null || echo "NOT_CONFIGURED"`

## Instructions
If the config above is NOT_CONFIGURED, ask the user:
- Which Slack channel?
- Paste a sample standup you liked
Then write the answers to ${CLAUDE_SKILL_DIR}/config.json.

Otherwise, post to the saved channel using the saved format.
```

The !`...` line runs as a shell command before Claude reads the prompt

Плохо: жёсткий скрипт. Хорошо: информация + свобода адаптации

Продумайте начальную настройку

Некоторые скиллы нуждаются в настройке с пользовательским контекстом. Например, скилл для постинга стендапа в Slack может спросить, в какой канал постить.

Хороший паттерн — хранить настройки в `config.json` в директории скилла. Если конфиг не настроен — агент спрашивает пользователя. Для структурированных вопросов с вариантами ответов — указать Claude использовать `AskUserQuestion` tool.

Поле Description — для модели

Когда Claude Code начинает сессию, он строит список всех доступных скиллов с их описаниями. По этому списку Claude решает «есть ли скилл для этого запроса?». Это значит, что поле description — **это не саммари, а описание КОГДА триггерить этот скилл**.

Description = когда триггерить, а не что внутри

⚠ **Дополнение Claude Opus 4.6 (Март 2026)**

Это критически важный совет. Разница между хорошим и плохим описанием: «Помогает ревьюить код» (~45 токенов, триггерит на всё) vs. «Use for PR reviews in Rust projects with focus on unsafe code and error handling» (~15 токенов, точный триггер). Описание должно содержать условия триггера, а не описание содержимого.

Память и хранение данных

Некоторые скиллы могут включать форму памяти, храня данные внутри. Это может быть append-only текстовый лог, JSON-файлы или даже SQLite база данных.

Например, скилл `standup-post` может хранить `standups.log` с каждым написанным постом — в следующий раз Claude читает свою историю и видит, что изменилось.

Данные в директории скилла могут удалиться при апгрейде. Используйте стабильную папку — Claude Code предоставляет `${PLUGIN_DATA_DIR}` как стабильную папку для данных плагина.

Храните скрипты и генерируйте код

Один из самых мощных инструментов, которые можно дать Claude — **код**. Скрипты и библиотеки позволяют Claude тратить свои ходы на композицию — решая, что делать дальше, а не воспроизводя boilerplate.

```
standup-post/SKILL.md

## Memory

Append each standup to ${CLAUDE_PLUGIN_DATA}/standups.log
after posting. This folder persists across skill upgrades.

---

On each run:
- read the log to see what changed since yesterday
- write today's entry after sending to Slack
```

Пример: helper-функции для data-science скилла

Например, в вашем data-science скилле можно предоставить библиотеку функций для получения данных. Claude затем генерирует скрипты на лету для более сложного анализа: «Что произошло во вторник?»

Скоупированные хуки

Skills могут включать хуки, которые активируются только при вызове скилла и длятся до конца сессии. Используйте для более строгих хуков, которые не нужны постоянно:

- `prod-guardrails` — блокирует rm -rf, DROP TABLE, force-push, kubectl delete. Нужно только при работе с prod — постоянное включение раздражает
- `scope-lock` — блокирует Edit/Write вне указанной директории. Полезно при дебаге: «хочу добавить логи, но случайно "чиню" несвязанный код»

```
lib/signups.py

def fetch(day):
    """Signups from events.raw for one day.
       - event='signup_completed', NOT 'signup_started'
       - dedupe by anonymous_id — user_id is null until after signup"""

def by_referrer(df):
    """Group by traffic source.
       - '(direct)' and '' and None all mean organic"""

def by_landing_page(df):
    """Group by entry page.
       - '/', '/index', '/home' are all the homepage
       - strips query params so UTM'd links collapse"""
```

Скоупированные хуки: активируются только при вызове скилла

⚠ **Дополнение Claude Opus 4.6 (Март 2026)**

Скоупированные хуки — мощный паттерн для безопасности в агентных пайплайнах. Каждая стадия вашего SDLC (Design, Dev, QA) может активировать свои собственные хуки: Dev-агент получает prod-guardrails, QA-агент получает scope-lock чтобы не менять код. Это реализация принципа least privilege для каждого агента.

Распространение Skills в команде

Одно из главных преимуществ Skills — возможность делиться ими с командой. Два способа:

- **Чекинить скиллы в репозиторий** (в `./.claude/skills`) — хорошо работает для небольших команд с несколькими репозиториями
- **Внутренний маркетплейс плагинов** — пользователи могут устанавливать нужные им плагины; масштабируется лучше, так как каждый скилл в репо добавляет немного контекста модели

Управление маркетплейсом

Как решать, какие скиллы попадают в маркетплейс? Мы не имеем централизованной команды. Вместо этого ищем самые полезные скиллы органически:

- Если у вас есть скилл, который хотите предложить — загрузите в sandbox-папку на GitHub и покажите в Slack
- Когда скилл набрал тягу (определяет автор) — PR для переноса в маркетплейс
- **Предупреждение:** легко создать плохие или дублирующие скиллы; убедитесь в курации перед релизом

Зависимости между скиллами

Скиллы могут зависеть друг от друга. Например, скилл загрузки файлов и скилл генерации CSV, который загружает результат. Управление зависимостями пока не встроено в маркетплейсы, но можно просто ссылаться на другие скиллы внутри SKILL.md.

Мониторинг использования

Для понимания эффективности скиллов мы используем **PreToolUse хук для логирования использования**. Это позволяет находить популярные скиллы или те, которые триггерятся реже ожидаемого.

```
investigate.py · generated by Claude

from lib.signups import fetch, by_referrer, by_landing_page

mon, tue = fetch("2024-03-11"), fetch("2024-03-12")

print(by_referrer(tue) - by_referrer(mon))      # organic -60%, paid flat
print(by_landing_page(tue) - by_landing_page(mon))  # homepage specifically

# → something broke on / on Tuesday
```

Мониторинг использования скиллов через PreToolUse хук

⚠ **Дополнение Claude Opus 4.6 (Март 2026)**

Мониторинг использования скиллов — критически важен для зрелых AI-agent систем. Рекомендация: добавьте логирование не только факта использования, но и результата (pass/fail). Это даст вам метрики эффективности каждого скилла и поможет понять, какие скиллы нуждаются в обновлении Gotchas-секции.

Заключение

Skills — невероятно мощные и гибкие инструменты для агентов, но мы все ещё в начале пути и выясняем, как использовать их наилучшим образом.

Думайте об этом скорее как о коллекции полезных советов, чем о законченном руководстве. Лучший способ понять скиллы — начать, экспериментировать и смотреть, что работает. Большинство наших скиллов начинались с нескольких строк и одного gotcha, и становились лучше потому, что люди продолжали дополнять их по мере того, как Claude учился.

⚠ **Дополнение Claude Opus 4.6 (Март 2026)**

Итого: план внедрения Skills для вашей команды:
1) Начните с 2–3 скиллов из категорий 2 (Verification) и 4 (Business Automation).
2) Добавьте секцию Gotchas после первой недели использования.
3) Используйте progressive disclosure — не грузите всё в SKILL.md.
4) Поле description = триггер, не описание содержимого.
5) Добавьте скоупированные хуки для prod-безопасности.
6) Логируйте использование через PreToolUse для метрик эффективности.

Оригинал: https://x.com/trq212/status/2033949937936085378
Автор: Thariq — https://x.com/trq212
Подготовил перевод: alienxs2 — https://github.com/alienxs2/cc_bestpractice_russian
Перевод и дополнения: Claude Opus 4.6 (Anthropic), Март 2026